

Diagonal Polynomial Laws in Odd Iterated Thue–Morse Summation

Riordan-array structure, 2-adic Bell recurrences, exact arithmetic,
and fast Wolfram Language evaluation

Vladimir Reshetnikov

ProveIt Fabius-function research program

3 September 2026

Abstract

Let $\varepsilon_j = (-1)^{s_2(j)}$ be the signed Thue–Morse sequence and let $S^{(r)} = \mathcal{P}^r \varepsilon$ denote r inclusive prefix summations. The table in the question is, after making its shift explicit,

$$s(n, k) = S^{(2n+1)}(k - n - 1) \quad (k \geq n + 1),$$

with $s(n, k) = 0$ for $k \leq n$. For a fixed diagonal offset m define $D_m(n) = s(n, n + 1 + m)$. The principal result is the formal generating identity

$$\sum_{m \geq 0} D_m(x) z^m = \frac{\mathcal{T}(z^2)}{(1 - z)^{2x}}, \quad \mathcal{T}(z) = \prod_{j \geq 0} (1 - z^{2^j}) = \sum_{q \geq 0} \varepsilon_q z^q,$$

which yields the exact polynomial formula

$$D_m(x) = \sum_{q=0}^{\lfloor m/2 \rfloor} \varepsilon_q \binom{2x + m - 2q - 1}{m - 2q}.$$

Thus the m th diagonal is a degree- m polynomial with leading coefficient $2^m/m!$. After shifting the column index, the complete two-dimensional table is the unit lower-triangular Riordan array

$$\left(\mathcal{T}(z^2), \frac{z}{(1 - z)^2} \right).$$

This observation is developed into a general subdiagonal theorem for proper Riordan arrays.

The logarithm of the diagonal generating function has the unusually simple 2-adic cumulants

$$\lambda_r(x) = 2x + 2 - 2^{\nu_2(r)+1},$$

so the polynomials satisfy a Newton–Bell recurrence, complete Bell-polynomial and partition formulas, and a determinant formula. They also form a Sheffer sequence for the half-step difference operator:

$$D_m(x) - D_m\left(x - \frac{1}{2}\right) = D_{m-1}(x).$$

The arithmetic is exact: $m!D_m(x)/2^{\lceil m/2 \rceil}$ is a primitive integer polynomial, giving the exact common denominator of every diagonal. Every rational zero is an integer or a half-integer. All nonnegative rational zeros are classified by a dyadic residue condition, which explains the visible factors x , $2x - 1$, $x - 1$, and their higher analogues. Negative half-grid values have a finite binomial–Thue–Morse formula.

Finally, the row factorization into finite geometric products proves the block repetition, reflection, complement, plateau, and zero-run rules used by the supplied Wolfram Language program. A new signed-prefix-moment recursion then evaluates an arbitrary table entry in $O(n^2 \log m)$ exact arithmetic operations, where $m = k - n - 1$, rather than summing $O(m)$ terms. Complete Wolfram Language and Python implementations accompany the article.

Contents

1	Scope, conventions, and repository context	4
2	From inclusive prefix sums to the table	4
2.1	The prefix operator and its convolution kernel	4
2.2	Exact identification of the code’s row and column indices	5
2.3	Row generating functions	6
3	Riordan-array structure and a general subdiagonal theorem	6
3.1	The shifted table is a proper Riordan array	6
3.2	Why fixed subdiagonals are always polynomials	7
3.3	General Bell and Sheffer descriptions	8
3.4	Application to the present table	8
4	Closed forms and polynomial anatomy	9
4.1	The finite rising-factorial formula	9
4.2	The first diagonals	10
4.3	All monomial coefficients	10
4.4	A basis over integer-valued polynomials	11
5	2-adic Newton–Bell structure and operational recurrences	11
5.1	Logarithmic cumulants	11
5.2	Bell-polynomial, partition, and determinant formulas	12
5.3	The half-step lowering law	13
6	Finite blocks, restricted compositions, symmetry, and plateaux	14
6.1	A universal finite block at nonnegative half-integers	14
6.2	The exact central plateau and complement law	15
6.3	Specialization to row n of the original table	15
7	Exact denominators, primitive normalization, and rational zeros	16
7.1	A primitive integer normalization	17
7.2	The exact common denominator	17
7.3	Every rational zero lies on the half-integer lattice	18
8	Half-integer values and factorization laws	18
8.1	Complete classification of nonnegative rational roots	18
8.2	Exact values on the negative half-grid	19
9	Fast exact evaluation	20
9.1	Symbolic generation of any diagonal	21
9.2	Direct exact evaluation	21
9.3	Logarithmic-time signed prefix moments	22
9.4	Memory and implementation notes	23
10	Connection with the Fabius and Rvachev approximation layer	23

11 Further deductions and research directions	24
11.1 Scaling regimes beyond fixed diagonals	24
11.2 Residual factors and zero geometry	25
11.3 Automatic and p -adic structure	25
11.4 Formalization opportunities	25
12 Summary of the main formulas	26
A Nonnegative rational factors through $m = 31$	26
B Complete Wolfram Language source	27
C Complete Python verification source	30
D Reproduction commands	36

1 Scope, conventions, and repository context

The `ProveIt` Fabius-function development contains an early note on iterated summation of signed Thue–Morse prefixes and a later Lean development that formalizes the exact convolution, generating-series, zero-run, and approximation statements [7, 8, 9]. It also contains a machine-checked audit that refutes several unrelated asymptotic and normalization claims in the early note [10]. The present article uses only the exact discrete structure. In particular, none of the refuted proxy or error estimates is used here.

The diagonal identities, Riordan-array interpretation, 2-adic Bell recurrence, primitive normalization, rational-root restriction, and fast prefix-moment evaluator below are derived directly from the formal power series of the table. They should therefore be read as an algebraic companion to the repository’s analytic Fabius approximation theory. No claim of literature priority is intended beyond this repository-level comparison.

Throughout,

$$\mathbb{N} = \{0, 1, 2, \dots\}, \quad \varepsilon_j = (-1)^{s_2(j)} = (-1)^{\text{popcount}(j)},$$

where $s_2(j)$ is the sum of the binary digits of j . We use the standard identities

$$\varepsilon_{2j} = \varepsilon_j, \quad \varepsilon_{2j+1} = -\varepsilon_j. \quad (1)$$

All generating-function identities are identities in a formal power-series ring over $\mathbb{Q}$ unless an analytic domain is stated explicitly. Consequently there are no convergence issues in coefficient extraction.

Indexing guide. The row parameter n in the code is not the number of prefix summations. Row n contains the $(2n + 1)$ -fold inclusive prefix sum, shifted right by $n + 1$ columns. The natural diagonal coordinate is

$$m = k - n - 1.$$

The polynomial $D_m(x)$ describes the line $k = n + 1 + m$; its value at the integer $x = n$ is exactly the table entry on that line.

2 From inclusive prefix sums to the table

2.1 The prefix operator and its convolution kernel

For a sequence $a = (a_0, a_1, \dots)$ define the inclusive prefix operator

$$(\mathcal{P}a)_r = \sum_{j=0}^r a_j, \quad r \in \mathbb{N}.$$

Let $S_r^{(0)} = \varepsilon_r$ and $S_r^{(\ell)} = \mathcal{P}^\ell \varepsilon$ for $\ell \geq 1$.

Proposition 2.1 (Iterated-prefix convolution). *For $\ell \geq 1$ and $r \geq 0$,*

$$S_r^{(\ell)} = \sum_{j=0}^r \binom{r-j+\ell-1}{\ell-1} \varepsilon_j. \quad (2)$$

Equivalently,

$$\sum_{r \geq 0} S_r^{(\ell)} z^r = \frac{\mathcal{T}(z)}{(1-z)^\ell}, \quad \mathcal{T}(z) := \sum_{j \geq 0} \varepsilon_j z^j. \quad (3)$$

Proof. One prefix summation multiplies an ordinary generating function by $(1 - z)^{-1}$:

$$\sum_{r \geq 0} \left(\sum_{j=0}^r a_j \right) z^r = \sum_{j \geq 0} a_j z^j \sum_{h \geq 0} z^h = \frac{1}{1-z} \sum_{j \geq 0} a_j z^j.$$

Iterating gives (3). The coefficient of z^{r-j} in $(1 - z)^{-\ell}$ is $\binom{r-j+\ell-1}{\ell-1}$, which gives (2). $\square$

The signed Thue–Morse generating series has the classical product

$$\mathcal{T}(z) = \prod_{j=0}^{\infty} (1 - z^{2^j}). \quad (4)$$

Indeed, choosing either 1 or $-z^{2^j}$ from each factor records the binary expansion of the exponent, with sign $(-1)^{s_2(j)}$. Separating the first factor gives the particularly useful functional identity

$$\mathcal{T}(z) = (1 - z)\mathcal{T}(z^2). \quad (5)$$

The product and its many consequences are standard in the theory of the Prouhet–Thue–Morse sequence [1, 2].

2.2 Exact identification of the code’s row and column indices

Definition 2.2 (The shifted odd-prefix table). For $n, k \in \mathbb{N}$ define

$$s(n, k) = \begin{cases} 0, & k \leq n, \\ S_{k-n-1}^{(2n+1)}, & k \geq n+1. \end{cases} \quad (6)$$

Proposition 2.3 (Agreement with the twofold-summation recurrence). For $n \geq 1$ and $k \geq 0$,

$$s(n, k) = \sum_{j=0}^{k-1} (k-j)s(n-1, j), \quad (7)$$

where an empty sum is zero.

Proof. For any sequence a ,

$$(\mathcal{P}^2 a)_N = \sum_{t=0}^N (N-t+1)a_t.$$

Take $a_t = S_t^{(2n-1)}$ and $N = k - n - 1$. In the sum on the right side of (7), all terms with $j \leq n-1$ vanish by definition. Writing $t = j - n$ in the remaining terms gives

$$\sum_{j=n}^{k-1} (k-j)S_{j-n}^{(2n-1)} = \sum_{t=0}^{k-n-1} (k-n-t)S_t^{(2n-1)} = S_{k-n-1}^{(2n+1)}.$$

This is $s(n, k)$ by (6). $\square$

The convolution formula now gives a direct expression for every table entry.

Corollary 2.4 (Single-sum formula for the complete table). For $k \geq n+1$,

$$s(n, k) = \sum_{j=0}^{k-n-1} \binom{k+n-1-j}{2n} \varepsilon_j. \quad (8)$$

Proof. Apply theorem 2.1 with $\ell = 2n+1$ and $r = k-n-1$. $\square$

2.3 Row generating functions

Let

$$G_n(z) = \sum_{k \geq 0} s(n, k) z^k.$$

The shift by $n + 1$, (3), and (5) give two equivalent forms:

$$G_n(z) = z^{n+1} \frac{\mathcal{T}(z)}{(1-z)^{2n+1}} \quad (9)$$

$$= z^{n+1} \frac{\mathcal{T}(z^2)}{(1-z)^{2n}}. \quad (10)$$

In particular,

$$\frac{G_n(z)}{G_{n-1}(z)} = \frac{z}{(1-z)^2} \quad (n \geq 1). \quad (11)$$

The cumulative recurrence in the original code is therefore the integrated form of a local second-difference law.

Corollary 2.5 (Local discrete equation). *With the convention $s(n, k) = 0$ for $k < 0$, one has, for $n \geq 1$,*

$$s(n, k) - 2s(n, k-1) + s(n, k-2) = s(n-1, k-1). \quad (12)$$

Proof. Multiply (11) by $(1-z)^2$ and compare coefficients of z^k . $\square$

Equation (12) fills a rectangular portion of the table in constant work per cell. It is often preferable to reevaluating the weighted sum in (7) for each new cell.

Summing (10) over all rows yields a compact bivariate series:

$$\sum_{n, k \geq 0} s(n, k) u^n z^k = \frac{z \mathcal{T}(z^2)}{1 - uz/(1-z)^2}. \quad (13)$$

This already displays the table as a geometric sequence of row-generating functions, with the entire Thue–Morse irregularity confined to the single factor $\mathcal{T}(z^2)$.

3 Riordan-array structure and a general subdiagonal theorem

3.1 The shifted table is a proper Riordan array

Shift the column index once and transpose the visual convention by setting

$$A_{r,n} = s(n, r+1), \quad r, n \geq 0. \quad (14)$$

Then $A_{r,n} = 0$ for $r < n$ and $A_{n,n} = 1$. From (10),

$$A_{r,n} = [z^r] \mathcal{T}(z^2) \left(\frac{z}{(1-z)^2} \right)^n. \quad (15)$$

Thus A is the proper ordinary Riordan array

$$A = \left(\mathcal{T}(z^2), \frac{z}{(1-z)^2} \right). \quad (16)$$

We use the convention that a Riordan pair (g, f) denotes the lower-triangular matrix $[z^r] g(z) f(z)^n$. Standard background can be found in [4, 5]; all facts needed below are also proved directly.

The compositional inverse of $f(z) = z/(1-z)^2$ is

$$\bar{f}(w) = \frac{1 + 2w - \sqrt{1 + 4w}}{2w} = \frac{\sqrt{1 + 4w} - 1}{\sqrt{1 + 4w} + 1} = w - 2w^2 + 5w^3 - 14w^4 + \dots \quad (17)$$

Consequently the inverse unitriangular matrix is the Riordan array

$$A^{-1} = \left(\frac{1}{\mathcal{T}(\bar{f}(z)^2)}, \bar{f}(z) \right). \quad (18)$$

This inverse is not needed to generate the table, but it shows that the table is an invertible change of basis over $\mathbb{Z}$: both f and $\bar{f}$ have integer coefficients and $\mathcal{T}(0) = 1$.

3.2 Why fixed subdiagonals are always polynomials

The polynomial phenomenon is not accidental and is not special to Thue–Morse. It is a general feature of proper Riordan arrays.

Theorem 3.1 (Polynomial subdiagonals of a proper Riordan array). *Let K be a characteristic-zero field, let $g(z) \in K[[z]]$ satisfy $g(0) = g_0 \neq 0$, and let $\phi(z) \in K[[z]]$ satisfy $\phi(0) = 1$. Form the proper Riordan array*

$$R_{r,n} = [z^r] g(z) (z\phi(z))^n.$$

For every fixed $m \geq 0$, define the formal continuation of the m th subdiagonal by

$$P_m(x) = [z^m] g(z) \phi(z)^x, \quad \phi(z)^x := \exp(x \log \phi(z)). \quad (19)$$

Then:

1. $P_m(x) \in K[x]$ and $\deg P_m \leq m$;
2. $P_m(n) = R_{n+m,n}$ for every $n \in \mathbb{N}$;
3. if $\phi_1 = [z]\phi(z) \neq 0$, then $\deg P_m = m$ and

$$[x^m] P_m(x) = \frac{g_0 \phi_1^m}{m!}. \quad (20)$$

Proof. Write $L(z) = \log \phi(z)$. Since $\phi(0) = 1$, one has $L(z) \in zK[[z]]$. Therefore

$$g(z) \phi(z)^x = g(z) \sum_{j \geq 0} \frac{x^j L(z)^j}{j!}.$$

Because $L(z)^j$ has order at least j , only $j \leq m$ can contribute to $[z^m]$. This proves polynomiality and the degree bound. At a nonnegative integer $x = n$, the formal power $\phi(z)^x$ is the ordinary n th power, so

$$P_m(n) = [z^m] g(z) \phi(z)^n = [z^{n+m}] g(z) (z\phi(z))^n = R_{n+m,n}.$$

Finally, the coefficient of x^m can only come from $j = m$. To reach z^m in $g(z)L(z)^m$, one must take the constant g_0 and the linear term $\phi_1 z$ from every copy of L . This gives $g_0 \phi_1^m / m!$. $\square$

Remark 3.2. The theorem gives a canonical polynomial, not merely an interpolation through integer points. Formula (19) defines it in the coefficient ring before any values are sampled. This distinction matters when one studies coefficients, denominators, roots, or values at half-integer arguments.

3.3 General Bell and Sheffer descriptions

The same setup gives a general “shape theorem” finer than degree alone. Write

$$\log\left(\frac{g(z)}{g_0}\right) + x \log \phi(z) = \sum_{r \geq 1} \kappa_r(x) \frac{z^r}{r}, \quad (21)$$

where each $\kappa_r(x)$ is affine in x . This normalized formal logarithm is defined over K because $g(z)/g_0$ has constant term one. Differentiating the generating series in z gives the Newton recurrence

$$mP_m(x) = \sum_{r=1}^m \kappa_r(x) P_{m-r}(x), \quad P_0(x) = g_0. \quad (22)$$

Equivalently, $m!P_m(x)/g_0$ is a complete exponential Bell polynomial in $(r-1)!\kappa_r(x)$. Thus the lower coefficients of every Riordan subdiagonal are controlled by the logarithmic coefficients of g and ϕ .

Moreover, with $Q_m(x) = m!P_m(x)$,

$$\sum_{m \geq 0} Q_m(x) \frac{z^m}{m!} = g(z) e^{xf(z)}, \quad f(z) = \log \phi(z). \quad (23)$$

Hence (Q_m) is a Sheffer sequence. If $f'(0) \neq 0$ and $q = f^{(-1)}$ is the compositional inverse of f , the associated delta operator is $q(D_x)$ and satisfies

$$q(D_x)Q_m(x) = mQ_{m-1}(x). \quad (24)$$

This is the finite-operator-calculus explanation of the especially simple half-step recurrence obtained later. See [6] for the general Sheffer formalism.

3.4 Application to the present table

For (16),

$$g(z) = \mathcal{T}(z^2), \quad \phi(z) = \frac{1}{(1-z)^2}, \quad \phi_1 = 2.$$

The m th Riordan subdiagonal is

$$A_{n+m,n} = s(n, n+m+1). \quad (25)$$

This motivates the central definition.

Definition 3.3 (Diagonal polynomial). For $m \geq 0$ define

$$D_m(x) = [z^m] \frac{\mathcal{T}(z^2)}{(1-z)^{2x}}. \quad (26)$$

Then

$$D_m(n) = s(n, n+1+m) \quad (n, m \in \mathbb{N}). \quad (27)$$

By theorem 3.1, D_m has exact degree m and leading coefficient $2^m/m!$. Summing (26) over integer n also gives a second bivariate form of the table:

$$\sum_{n, m \geq 0} D_m(n) u^n z^m = \frac{\mathcal{T}(z^2)(1-z)^2}{(1-z)^2 - u}. \quad (28)$$

Since $D_m(n)$ is degree m in n , its ordinary generating function in n is rational with denominator dividing $(1-u)^{m+1}$. Equation (28) packages all of these rational functions at once.

4 Closed forms and polynomial anatomy

4.1 The finite rising-factorial formula

Theorem 4.1 (General polynomial for a prescribed diagonal). *For every $m \geq 0$,*

$$D_m(x) = \sum_{q=0}^{\lfloor m/2 \rfloor} \varepsilon_q \binom{2x + m - 2q - 1}{m - 2q} \quad (29)$$

$$= \sum_{q=0}^{\lfloor m/2 \rfloor} \varepsilon_q \frac{(2x)^{\overline{m-2q}}}{(m-2q)!} \quad (30)$$

$$= \sum_{j=0}^m \varepsilon_j \binom{2x + m - j}{m - j}. \quad (31)$$

The convention is $y^{\overline{0}} = 1$. In particular, $D_m \in \mathbb{Q}[x]$, $\deg D_m = m$, and

$$[x^m]D_m(x) = \frac{2^m}{m!}. \quad (32)$$

Proof. Expand

$$\mathcal{T}(z^2) = \sum_{q \geq 0} \varepsilon_q z^{2q}, \quad (1-z)^{-2x} = \sum_{p \geq 0} \binom{2x + p - 1}{p} z^p.$$

Extracting z^m proves (29); the generalized-binomial identity $\binom{2x+p-1}{p} = (2x)^{\overline{p}}/p!$ gives (30). Alternatively, (5) says $\mathcal{T}(z^2) = \mathcal{T}(z)/(1-z)$, so

$$\frac{\mathcal{T}(z^2)}{(1-z)^{2x}} = \frac{\mathcal{T}(z)}{(1-z)^{2x+1}}.$$

Expanding both factors gives (31). Only the $q = 0$ term of (30) has degree m , and its leading term is $(2x)^m/m!$. $\square$

Formula (30) is the requested general shape. It expresses a whole diagonal as a short signed combination of rising-factorial basis polynomials whose degrees descend by two. The coefficients are not fitted from data: they are exactly the signed Thue–Morse symbols. Written directly in the indexing of the question, for every fixed diagonal distance $d \geq 1$,

$$s(n, n+d) = \sum_{q=0}^{\lfloor (d-1)/2 \rfloor} \varepsilon_q \frac{(2n)^{\overline{d-1-2q}}}{(d-1-2q)!}. \quad (33)$$

This is a polynomial of degree $d-1$ in n , with leading coefficient $2^{d-1}/(d-1)!$.

4.2 The first diagonals

The first eight polynomials are

$$\begin{aligned}
D_0(x) &= 1, \\
D_1(x) &= 2x, \\
D_2(x) &= (x+1)(2x-1), \\
D_3(x) &= \frac{2}{3}x(x+2)(2x-1), \\
D_4(x) &= \frac{1}{6}(4x^4 + 12x^3 - x^2 - 3x - 6), \\
D_5(x) &= \frac{1}{15}x(x-1)(4x^3 + 24x^2 + 39x + 34), \\
D_6(x) &= \frac{1}{90}(x-1)(x+1)(2x-1)(4x^3 + 32x^2 + 75x + 90), \\
D_7(x) &= \frac{1}{315}x(x-1)(x+2)(2x-1)(4x^3 + 40x^2 + 123x + 192).
\end{aligned}$$

Substituting $x = n$ and $m = k - n - 1$ reproduces exactly the displayed initial rules in the question. The next polynomial, included to illustrate that the factor pattern is genuinely irregular, is

$$D_8(x) = \frac{x+1}{2520} \begin{pmatrix} 16x^7 + 208x^6 + 856x^5 + 1384x^4 \\ -1055x^3 - 3719x^2 + 3570x - 2520 \end{pmatrix}. \quad (34)$$

The later sections explain every nonnegative rational linear factor and a substantial family of negative integer factors.

4.3 All monomial coefficients

Let $\left[\begin{smallmatrix} p \\ r \end{smallmatrix} \right]$ denote the unsigned Stirling number of the first kind, defined by

$$y^{\bar{p}} = \sum_{r=0}^p \left[\begin{smallmatrix} p \\ r \end{smallmatrix} \right] y^r. \quad (35)$$

Then (30) immediately gives the complete coefficient formula.

Corollary 4.2 (Monomial coefficients). *For $0 \leq r \leq m$,*

$$[x^r]D_m(x) = 2^r \sum_{q=0}^{\lfloor (m-r)/2 \rfloor} \varepsilon_q \frac{\left[\begin{smallmatrix} m-2q \\ r \end{smallmatrix} \right]}{(m-2q)!}. \quad (36)$$

The first three terms at the top of the polynomial are especially simple. For fixed $m \geq 2$,

$$\begin{aligned}
D_m(x) &= \frac{2^m}{m!}x^m + \frac{2^{m-2}}{(m-2)!}x^{m-1} \\
&\quad + \frac{2^{m-2}(3m^2 - 7m - 22)}{24(m-2)!}x^{m-2} + \mathcal{O}(x^{m-3}).
\end{aligned} \quad (37)$$

Here terms with negative powers are simply omitted for small m .

Proof. The coefficient of x^{m-1} comes only from $q = 0$ and uses $\left[\begin{smallmatrix} m \\ m-1 \end{smallmatrix} \right] = \binom{m}{2}$. For x^{m-2} , the $q = 0$ and $q = 1$ terms contribute. The standard near-diagonal Stirling value

$$\left[\begin{smallmatrix} m \\ m-2 \end{smallmatrix} \right] = \frac{m(m-1)(m-2)(3m-1)}{24}$$

gives (37) after simplification. $\square$

Consequently, for a fixed diagonal offset m and $n \rightarrow \infty$,

$$D_m(n) = \frac{2^m n^m}{m!} \left(1 + \frac{m(m-1)}{4n} + \frac{m(m-1)(3m^2 - 7m - 22)}{96n^2} + \mathcal{O}(n^{-3}) \right). \quad (38)$$

This is an ordinary polynomial asymptotic, not an approximation to the whole row. It describes a fixed number of cells from the left edge while the row order grows.

4.4 A basis over integer-valued polynomials

The rational monomial coefficients obscure an important integrality fact. Put

$$U(z) = \frac{1}{(1-z)^2} - 1 = 2z + 3z^2 + 4z^3 + \dots$$

Then

$$(1-z)^{-2x} = (1+U(z))^x = \sum_{r \geq 0} \binom{x}{r} U(z)^r.$$

Therefore

$$D_m(x) = \sum_{r=0}^m C_{m,r} \binom{x}{r}, \quad C_{m,r} = [z^m] \mathcal{T}(z^2) U(z)^r \in \mathbb{Z}. \quad (39)$$

In particular $C_{m,m} = 2^m$, and $D_m(a) \in \mathbb{Z}$ for every integer a , positive or negative. The first few rows $(C_{m,0}, \dots, C_{m,m})$ are

m	binomial-basis coefficients
0	1
1	0, 2
2	-1, 3, 4
3	0, 2, 12, 8
4	-1, 2, 21, 36, 16
5	0, 0, 32, 94, 96, 32
6	1, -1, 41, 195, 328, 240, 64

The low-index coefficients retain the signed Thue–Morse oscillation, while the upper part is governed by the positive Pascal-type factor $(1-z)^{-2x}$.

5 2-adic Newton–Bell structure and operational recurrences

5.1 Logarithmic cumulants

The lower coefficients of D_m are controlled by a remarkably sparse 2-adic rule. Taking the logarithm of (26),

$$\begin{aligned} \log \left(\frac{\mathcal{T}(z^2)}{(1-z)^{2x}} \right) &= -2x \log(1-z) + \sum_{j \geq 1} \log(1-z^{2^j}) \\ &= \sum_{r \geq 1} \lambda_r(x) \frac{z^r}{r}. \end{aligned}$$

For a fixed r , the product part receives contributions precisely from $2^j \mid r$ with $1 \leq j \leq \nu_2(r)$. Hence

$$\lambda_r(x) = 2x + 2 - 2^{\nu_2(r)+1}. \quad (40)$$

The beginning of the sequence is

$$2x, \quad 2x - 2, \quad 2x, \quad 2x - 6, \quad 2x, \quad 2x - 2, \quad 2x, \quad 2x - 14, \dots$$

Only the positions divisible by a large power of two deviate substantially from $2x$.

Theorem 5.1 (2-adic Newton recurrence). *The diagonal polynomials are uniquely generated by*

$$D_0(x) = 1, \quad mD_m(x) = \sum_{r=1}^m \left(2x + 2 - 2^{\nu_2(r)+1}\right) D_{m-r}(x). \quad (41)$$

Proof. Let $F(x, z) = \sum_{m \geq 0} D_m(x) z^m$. From $\log F = \sum_{r \geq 1} \lambda_r z^r / r$,

$$\frac{\partial F}{\partial z} = F \sum_{r \geq 1} \lambda_r(x) z^{r-1}.$$

Comparing coefficients of z^{m-1} gives (41). $\square$

This recurrence is well suited to symbolic generation: it uses only addition, multiplication by an affine polynomial, and division by the current index. It also makes visible a structure that is almost invisible in the expanded polynomials: all irregularity enters through $\nu_2(r)$.

5.2 Bell-polynomial, partition, and determinant formulas

Let $B_m(y_1, \dots, y_m)$ be the complete exponential Bell polynomial, defined by

$$\exp\left(\sum_{r \geq 1} y_r \frac{z^r}{r!}\right) = \sum_{m \geq 0} B_m(y_1, \dots, y_m) \frac{z^m}{m!}.$$

Then

$$D_m(x) = \frac{1}{m!} B_m(0! \lambda_1(x), 1! \lambda_2(x), \dots, (m-1)! \lambda_m(x)). \quad (42)$$

Equivalently, if a partition $\mu \vdash m$ has a_r parts equal to r and

$$z_\mu = \prod_{r \geq 1} r^{a_r} a_r!,$$

then

$$D_m(x) = \sum_{\mu \vdash m} \frac{1}{z_\mu} \prod_{r \geq 1} \lambda_r(x)^{a_r}. \quad (43)$$

Thus every coefficient is a finite weighted sum indexed by integer partitions, with the weight of a part determined only by its 2-adic valuation.

A determinant form follows from the standard Newton determinant:

$$m! D_m(x) = \det \begin{pmatrix} \lambda_1 & -1 & 0 & \cdots & 0 \\ \lambda_2 & \lambda_1 & -2 & \ddots & \vdots \\ \lambda_3 & \lambda_2 & \lambda_1 & \ddots & 0 \\ \vdots & \vdots & \vdots & \ddots & -(m-1) \\ \lambda_m & \lambda_{m-1} & \lambda_{m-2} & \cdots & \lambda_1 \end{pmatrix}, \quad (44)$$

where $\lambda_r = \lambda_r(x)$. Formulas (42)–(44) are equivalent descriptions of the same logarithmic data; they are useful in different symbolic systems.

5.3 The half-step lowering law

The Sheffer structure becomes unusually concrete because

$$f(z) = \log(1 - z)^{-2} = -2 \log(1 - z), \quad f^{(-1)}(t) = 1 - e^{-t/2}.$$

The operator $e^{-D_x/2}$ translates a polynomial by $-1/2$. Therefore the delta operator $1 - e^{-D_x/2}$ is a backward half-step difference.

Theorem 5.2 (Discrete Appell law). *For $m \geq 1$,*

$$D_m(x) - D_m\left(x - \frac{1}{2}\right) = D_{m-1}(x). \quad (45)$$

Equivalently, for $P_m(x) = m!D_m(x)$,

$$\left(1 - e^{-D_x/2}\right) P_m(x) = mP_{m-1}(x).$$

Proof. Replacing x by $x - 1/2$ in (26) multiplies the generating series by $1 - z$:

$$\sum_{m \geq 0} D_m\left(x - \frac{1}{2}\right) z^m = (1 - z) \sum_{m \geq 0} D_m(x) z^m.$$

Coefficient comparison gives (45). □

Several useful identities follow immediately.

Corollary 5.3 (Half-grid translation formulas). *For every integer $a \geq 0$,*

$$D_m\left(x - \frac{a}{2}\right) = \sum_{j=0}^{\min(a,m)} (-1)^j \binom{a}{j} D_{m-j}(x), \quad (46)$$

$$D_m\left(x + \frac{1}{2}\right) = \sum_{j=0}^m D_j(x), \quad (47)$$

$$D_m(x + 1) = \sum_{j=0}^m (m - j + 1) D_j(x). \quad (48)$$

Proof. The first identity follows by iterating multiplication of the generating series by $1 - z$. A positive half-step multiplies it by $(1 - z)^{-1}$, and a unit step by $(1 - z)^{-2}$, giving the other two coefficient convolutions. □

The full addition law is

$$D_m(x + y) = \sum_{j=0}^m D_j(x) \binom{2y + m - j - 1}{m - j}. \quad (49)$$

This is the Sheffer addition formula with associated basic sequence $(2y)^{\bar{r}}$.

Differentiation in x gives another triangular recurrence.

Corollary 5.4 (Differential recurrence). *For $m \geq 1$,*

$$D'_m(x) = 2 \sum_{r=1}^m \frac{D_{m-r}(x)}{r}. \quad (50)$$

Proof. Differentiate (26) with respect to x and use $-2 \log(1 - z) = 2 \sum_{r \geq 1} z^r / r$. □

Finally, applying the local table equation (12) along a subdiagonal gives the integer-step relation

$$D_m(n) - D_m(n - 1) = 2D_{m-1}(n) - D_{m-2}(n), \quad (51)$$

with $D_{-1} = D_{-2} = 0$. This is also obtained by applying the half-step law twice.

6 Finite blocks, restricted compositions, symmetry, and plateaux

The same factorization that classifies half-integer values also explains every large-scale branch in the supplied Wolfram Language definition.

6.1 A universal finite block at nonnegative half-integers

For an integer $a \geq 0$, define

$$B_a = 2^{a+1}, \quad (52)$$

$$H_a(z) = \prod_{j=1}^a \left(1 + z + \cdots + z^{2^j-1}\right), \quad (53)$$

$$\delta_a = \deg H_a = B_a - a - 2. \quad (54)$$

The empty product H_0 is 1. Write

$$H_a(z) = \sum_{r=0}^{\delta_a} h_{a,r} z^r.$$

After extending $h_{a,r} = 0$ outside $0 \leq r \leq \delta_a$.

Theorem 6.1 (Finite-block factorization). *For every $a \geq 0$,*

$$\sum_{m \geq 0} D_m\left(\frac{a}{2}\right) z^m = H_a(z) \mathcal{T}(z^{B_a}). \quad (55)$$

Hence, if $m = qB_a + r$ with $0 \leq r < B_a$, then

$$D_m\left(\frac{a}{2}\right) = \varepsilon_q h_{a,r}. \quad (56)$$

Moreover:

1. $h_{a,r} > 0$ for $0 \leq r \leq \delta_a$;
2. $h_{a,r} = h_{a,\delta_a-r}$;
3. the final $a + 1$ residues $r = B_a - a - 1, \dots, B_a - 1$ have coefficient zero.

Proof. At $x = a/2$, (26) becomes $\mathcal{T}(z^2)/(1-z)^a$. Split the factors of $\mathcal{T}(z^2)$ at exponent 2^a :

$$\begin{aligned} \frac{\mathcal{T}(z^2)}{(1-z)^a} &= \prod_{j=1}^a \frac{1-z^{2^j}}{1-z} \prod_{j=a+1}^{\infty} (1-z^{2^j}) \\ &= H_a(z) \mathcal{T}(z^{2^{a+1}}). \end{aligned}$$

This proves (55). Since $\deg H_a = B_a - a - 2 < B_a$, shifted copies of H_a do not overlap, proving (56).

Each factor of H_a has all coefficients equal to one on a consecutive interval. Repeated convolution therefore gives a positive coefficient at every degree from zero through the total degree. Each factor is palindromic, so their product is palindromic. The remaining assertions follow from the degree. $\square$

There is a direct combinatorial interpretation:

$$h_{a,r} = \# \left\{ (u_1, \dots, u_a) \in \mathbb{Z}^a : 0 \leq u_j < 2^j, \sum_{j=1}^a u_j = r \right\}. \quad (57)$$

Thus a complete nonzero block consists, up to the Thue–Morse sign, of the rank numbers of a product of finite chains. In particular, it is symmetric. It is also unimodal; one may prove this by repeated convolution of symmetric unimodal interval sequences. Stronger log-concavity statements can be obtained from standard convolution results, but unimodality is all that is needed here.

6.2 The exact central plateau and complement law

Set $M_a = 2^a = B_a/2$ and

$$C_a = H_{a-1}(1) = 2^{1+2+\dots+(a-1)} = 2^{\binom{a}{2}}, \quad (58)$$

with $C_0 = 1$. Split off the final factor:

$$H_a(z) = H_{a-1}(z)(1 + z + \dots + z^{M_a-1}).$$

The degree of H_{a-1} is $M_a - a - 1$, strictly less than M_a . Consequently, in the first half of the product, $h_{a,r}$ is a cumulative sum of the coefficients of H_{a-1} .

Theorem 6.2 (Plateau and complement). *For $a \geq 1$:*

1. *the maximum coefficient is C_a ;*
2. *the complete plateau is*

$$h_{a,r} = C_a \iff M_a - a - 1 \leq r \leq M_a - 1; \quad (59)$$

3. *for $0 \leq r \leq M_a - 1$,*

$$h_{a,r} + h_{a,M_a-a-2-r} = C_a, \quad (60)$$

where a coefficient with a negative subscript is interpreted as zero.

Proof. Write $H_{a-1}(z) = \sum_{j=0}^d p_j z^j$ with $d = M_a - a - 1$. For $0 \leq r \leq M_a - 1$,

$$h_{a,r} = \sum_{j=0}^{\min(r,d)} p_j.$$

The total sum is $\sum_{j=0}^d p_j = H_{a-1}(1) = C_a$, so the coefficient reaches C_a exactly when $r \geq d$. This proves the stated plateau in the first half; palindromicity shows that it is the complete plateau.

For the complement identity, put $r' = M_a - a - 2 - r = d - 1 - r$. When $r' < 0$, the first term has already reached C_a . Otherwise, palindromicity $p_j = p_{d-j}$ gives

$$C_a - h_{a,r'} = \sum_{j=r'+1}^d p_j = \sum_{j=0}^{d-r'-1} p_j = \sum_{j=0}^r p_j = h_{a,r}.$$

□

6.3 Specialization to row n of the original table

Take $a = 2n$ and abbreviate

$$B = 2^{2n+1}, \quad M = 2^{2n}, \quad C = 2^{\binom{2n}{2}} = 2^{n(2n-1)}. \quad (61)$$

The diagonal-offset form is particularly simple. If $m = qB + r$, $0 \leq r < B$, then

$$D_m(n) = \varepsilon_q h_{2n,r}. \quad (62)$$

Since $s(n, k) = D_{k-n-1}(n)$, this already evaluates the whole row. To match the code's block boundaries in the unshifted column k , define

$$c_{n,r} = [z^r] z^{n+1} H_{2n}(z), \quad 0 \leq r < B. \quad (63)$$

Then, for $k = qB + r$,

$$s(n, k) = \varepsilon_q c_{n,r}. \quad (64)$$

The finite core $c_{n,r}$ has the following exact geometry:

$$c_{n,r} = 0 \quad (0 \leq r \leq n \text{ or } B - n \leq r < B), \quad (65)$$

$$c_{n,r} = c_{n,B-r} \quad (0 \leq r \leq B), \quad (66)$$

$$c_{n,r} + c_{n,M-r} = C \quad (0 \leq r \leq M), \quad (67)$$

$$c_{n,r} = C \quad (M - n \leq r \leq M + n). \quad (68)$$

For (66), set $c_{n,B} = c_{n,0} = 0$.

These four formulas are exactly the mathematical content of the three large index-reduction branches in the supplied program:

- (64) gives signed repetition by complete blocks;
- (66) gives reflection in the half-block;
- (67) gives the quarter-block complement;
- (68) explains the constant $2^{\binom{2n}{2}}$.

The apparent asymmetry of the leading and trailing zero intervals inside one residue block is caused by the shift $n + 1$. Across a block boundary they join into an exact run of $2n + 1$ zeros, the expected Prouhet cancellation length for an order- $(2n + 1)$ prefix row.

Example 6.3 (The first nontrivial finite blocks). For $n = 1$, $B = 8$ and

$$H_2(z) = (1 + z)(1 + z + z^2 + z^3) = 1 + 2z + 2z^2 + 2z^3 + z^4.$$

Thus the diagonal-offset block is

$$1, 2, 2, 2, 1, 0, 0, 0,$$

followed by its negative, then by blocks with signs $-1, +1, \dots$ according to ε_q . For $n = 2$, $B = 32$ and the plateau height is $2^{\binom{4}{2}} = 64$, attained at diagonal offsets 11, 12, 13, 14, 15. (The final offsets 27, 28, 29, 30, 31 are instead the zero window.)

A small portion of the diagonal-coordinate table $(D_m(n))_{n,m \geq 0}$ is displayed below. Rows $n = 0$ and $n = 1$ already show the signed block repetition, while larger rows have not yet reached their first plateau within the displayed window.

$n \backslash m$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	0	-1	0	-1	0	1	0	-1	0	1	0	1	0	-1	0
1	1	2	2	2	1	0	0	0	-1	-2	-2	-2	-1	0	0	0
2	1	4	9	16	24	32	40	48	55	60	63	64	64	64	64	64
3	1	6	20	50	104	190	316	490	719	1008	1360	1776	2256	2800	3408	4080
4	1	8	35	112	293	664	1351	2528	4424	7328	11592	17632	25928	37024	51528	70112

7 Exact denominators, primitive normalization, and rational zeros

The polynomial formula has surprisingly rigid arithmetic. Although the coefficients of D_m are generally rational, one power of two accounts for the entire coefficient content after multiplication by $m!$.

7.1 A primitive integer normalization

Put

$$c_m = \left\lceil \frac{m}{2} \right\rceil, \quad Q_m(x) = \frac{m!}{2^{c_m}} D_m(x). \quad (69)$$

Theorem 7.1 (Primitive normalization). *For every $m \geq 0$,*

$$Q_m(x) \in \mathbb{Z}[x] \quad \text{and} \quad \text{content}(Q_m) = 1. \quad (70)$$

Its leading coefficient is

$$[x^m]Q_m(x) = 2^{m-c_m} = 2^{\lfloor m/2 \rfloor}. \quad (71)$$

Proof. Consider the q th summand of $m!D_m$ in (30), and put $p = m - 2q$. Pairing consecutive factors gives

$$(2x)^{\overline{p}} = \prod_{j=0}^{p-1} (2x + j) \in 2^{\lceil p/2 \rceil} \mathbb{Z}[x],$$

because each pair $(2x + 2j)(2x + 2j + 1)$ contributes a factor 2, and an unpaired final even factor contributes one more. The integer $m!/p! = (p+1)(p+2) \cdots m$ is a product of $2q$ consecutive integers and is divisible by at least 2^q . Therefore the whole summand is coefficientwise divisible by

$$2^{\lceil p/2 \rceil + q} = 2^{\lceil (m-2q)/2 \rceil + q} = 2^{c_m}.$$

This proves $Q_m \in \mathbb{Z}[x]$.

It remains to prove primitivity. The coefficient of x^{c_m} in the $q = 0$ summand of Q_m is the unsigned Stirling number $\begin{bmatrix} m \\ c_m \end{bmatrix}$. This number is odd: modulo 2,

$$x^{\overline{m}} = \prod_{j=0}^{m-1} (x + j) \equiv x^{c_m} (x + 1)^{\lfloor m/2 \rfloor},$$

so the coefficient of x^{c_m} is 1. For $q \geq 1$, the coefficient of x^{c_m} in $(2x)^{\overline{m-2q}}/2^{c_m-q}$ carries an additional factor 2^q whenever that degree exists, and is therefore even. Hence $[x^{c_m}]Q_m$ is odd.

The leading coefficient follows from (32) and is a pure power of two. Any common divisor of all coefficients must therefore be a power of two, but it must also divide the odd coefficient $[x^{c_m}]Q_m$. The content is consequently one. $\square$

7.2 The exact common denominator

Let $\text{den}(P)$ denote the least positive integer d such that $dP \in \mathbb{Z}[x]$.

Corollary 7.2 (Exact denominator). *For every $m \geq 0$,*

$$\boxed{\text{den}(D_m) = \frac{m!}{2^{\min(\nu_2(m!), \lceil m/2 \rceil)}}.} \quad (72)$$

Using Legendre's identity $\nu_2(m!) = m - s_2(m)$, this is completely explicit in the binary digits of m . For $m \geq 4$ it simplifies to

$$\text{den}(D_m) = \frac{m!}{2^{\lceil m/2 \rceil}}. \quad (73)$$

Proof. By theorem 7.1, $D_m = (2^{c_m}/m!)Q_m$ with Q_m primitive. Reduce the scalar $2^{c_m}/m!$ to lowest terms. Because Q_m has content one, no remaining prime in the denominator can cancel from every coefficient. This gives (72). The inequality $\nu_2(m!) \geq \lceil m/2 \rceil$ for $m \geq 4$ follows directly by counting even factors, with the small boundary cases checked separately. $\square$

The first values are

m	0	1	2	3	4	5	6	7
$\text{den}(D_m)$	1	1	1	3	6	15	90	315

m	8	9	10	11	12	13	14	15
$\text{den}(D_m)$	2520	11340	113400	623700	7484400	48648600	681080400	5108103000

This theorem explains why the seemingly ad hoc denominators in the displayed Wolfram Language rules follow a uniform law.

7.3 Every rational zero lies on the half-integer lattice

Theorem 7.3 (Rational-root restriction). *If $\alpha \in \mathbb{Q}$ and $D_m(\alpha) = 0$, then*

$$\alpha \in \frac{1}{2}\mathbb{Z}. \quad (74)$$

Proof. The zero set of D_m is the zero set of the primitive integer polynomial Q_m . By the rational-root theorem and (71), the reduced denominator of a nonzero rational root is a power of two. Write such a root as $p/2^e$ with p odd.

Let $Q_m(x) = \sum_{r=0}^m a_r x^r$. For $r \geq c_m$, every coefficient of x^r in $m!D_m$ is divisible by 2^r , because the coefficient of x^r in every rising factorial $(2x)^{\bar{p}}$ is divisible by 2^r . Hence

$$2^{r-c_m} \mid a_r \quad (r \geq c_m), \quad (75)$$

and equality holds in valuation for the leading coefficient $a_m = 2^{m-c_m}$.

Assume $e \geq 2$ and multiply $Q_m(p/2^e) = 0$ by 2^{em} . The leading term has 2-adic valuation $m - c_m$. For $c_m \leq r < m$, the r th term has valuation at least

$$(r - c_m) + e(m - r) = m - c_m + (e - 1)(m - r) > m - c_m.$$

For $r < c_m$, its valuation is at least $e(m - r) \geq 2(m - c_m + 1) > m - c_m$. Thus the leading term is the unique term of least 2-adic valuation, so the sum cannot vanish. This contradiction shows $e \leq 1$. $\square$

The theorem reduces the classification of rational linear factors to exact values on the half-integer lattice. The nonnegative half is completely classified next; the negative half has an exact finite test.

8 Half-integer values and factorization laws

8.1 Complete classification of nonnegative rational roots

The finite-block theorem gives an exact residue test.

Theorem 8.1 (Nonnegative half-integer zero criterion). *Let $m, a \in \mathbb{N}$ and put $B_a = 2^{a+1}$. Then*

$$D_m\left(\frac{a}{2}\right) = 0 \iff m \bmod B_a \in \{B_a - a - 1, \dots, B_a - 1\} \quad (76)$$

$$\iff B_a \mid m + j \text{ for some } 1 \leq j \leq a + 1. \quad (77)$$

When the value is nonzero and $m = qB_a + r$, its sign is ε_q .

Proof. By (56), the value is $\varepsilon_q h_{a,r}$ for $r = m \bmod B_a$. The coefficient $h_{a,r}$ is positive through degree $B_a - a - 2$ and zero in the final $a + 1$ positions. This proves (76). The last $a + 1$ residues are precisely the integers that become divisible by B_a after adding one of $1, \dots, a + 1$, proving (77). $\square$

Define the finite set

$$\mathcal{A}_m = \{a \in \mathbb{N} : m \bmod 2^{a+1} \geq 2^{a+1} - a - 1\}. \quad (78)$$

Then the squarefree product of the nonnegative rational linear factors is visible in

$$R_m^+(x) = \prod_{a \in \mathcal{A}_m} (2x - a), \quad R_m^+(x) \mid D_m(x) \quad \text{in } \mathbb{Q}[x]. \quad (79)$$

Conversely, by [theorem 7.3](#), this product contains one copy of the factor belonging to every nonnegative rational root of D_m . Thus (79) completely classifies the distinct nonnegative rational roots. It does not assert that every such zero is simple.

For $m \geq 1$, a root with parameter $a \geq 1$ implies $m \geq 2^{a+1} - a - 1 \geq 2^a$. Hence

$$\#\mathcal{A}_m \leq 1 + \lfloor \log_2 m \rfloor. \quad (80)$$

Only logarithmically many nonnegative rational factors can occur in a degree m polynomial.

The first few residue laws are particularly transparent:

$$\begin{aligned} x = 0 : \quad & m \equiv 1 \pmod{2}, \\ x = \frac{1}{2} : \quad & m \equiv 2, 3 \pmod{4}, \\ x = 1 : \quad & m \equiv 5, 6, 7 \pmod{8}, \\ x = \frac{3}{2} : \quad & m \equiv 12, 13, 14, 15 \pmod{16}, \\ x = 2 : \quad & m \equiv 27, 28, 29, 30, 31 \pmod{32}. \end{aligned}$$

These rules explain, for example, why x divides every odd-indexed D_m , why $2x - 1$ divides D_m exactly for $m \equiv 2, 3 \pmod{4}$, and why $x - 1$ first appears for $m = 5, 6, 7$.

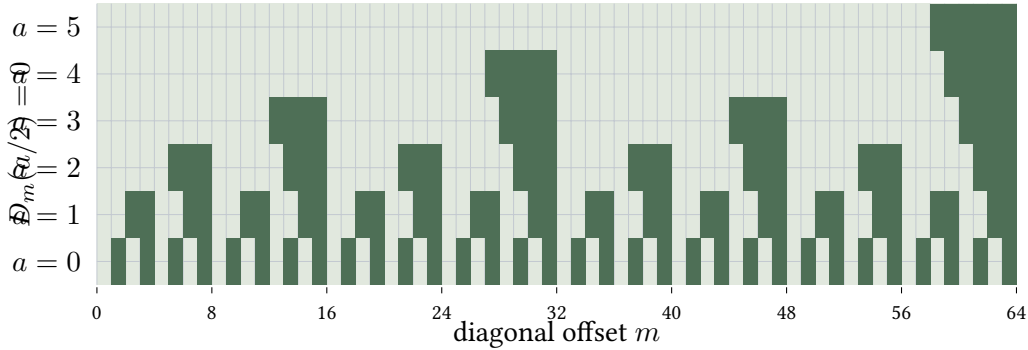


Figure 1: The exact nonnegative half-integer root raster for $0 \leq m < 64$ and $0 \leq a \leq 5$. A dark cell means that $2x - a$ divides $D_m(x)$. Row a consists of the final $a + 1$ residues in each period 2^{a+1} .

8.2 Exact values on the negative half-grid

A negative half-step is finite because it multiplies the generating function by a polynomial. Taking $x = -b/2$ in (26) gives

$$\sum_{m \geq 0} D_m \left(-\frac{b}{2} \right) z^m = (1 - z)^b \mathcal{T}(z^2). \quad (81)$$

Theorem 8.2 (Negative half-grid formula). *For $b, m \in \mathbb{N}$,*

$$D_m \left(-\frac{b}{2} \right) = \sum_{\substack{0 \leq j \leq \min(b, m) \\ j \equiv m \pmod{2}}} (-1)^j \binom{b}{j} \varepsilon_{(m-j)/2}. \quad (82)$$

Consequently, together with [theorem 8.1](#), equation (82) gives a finite exact decision procedure for every rational root of every D_m .

Proof. Expand $(1 - z)^b$ and $\mathcal{T}(z^2) = \sum_{q \geq 0} \varepsilon_q z^{2q}$ in (81). A term contributes to z^m exactly when $m - j = 2q \geq 0$. $\square$

Some low negative arguments simplify sharply. In the next corollary only, extend the notation by $\varepsilon_j = 0$ for $j < 0$.

Corollary 8.3 (Selected negative factors). *For $m \geq 0$ and $q \geq 1$,*

$$D_m\left(-\frac{1}{2}\right) = \varepsilon_m, \quad (83)$$

$$D_{2q}(-1) = \varepsilon_q + \varepsilon_{q-1}, \quad D_{2q+1}(-1) = -2\varepsilon_q, \quad (84)$$

$$D_{2q}(-2) = \varepsilon_q + 6\varepsilon_{q-1} + \varepsilon_{q-2}, \quad D_{2q+1}(-2) = -4(\varepsilon_q + \varepsilon_{q-1}). \quad (85)$$

In particular,

$$(x + 1) \mid D_{2q}(x) \iff \nu_2(q) \text{ is even}, \quad (86)$$

$$(x + 2) \mid D_{2q+1}(x) \iff \nu_2(q) \text{ is even}. \quad (87)$$

The values $D_{2q+1}(-1)$ and $D_{2q}(-2)$ never vanish.

Proof. Equation (83) follows because $(1 - z)\mathcal{T}(z^2) = \mathcal{T}(z)$. Equations (84) and (85) follow by multiplying $\mathcal{T}(z^2)$ by $(1 - z)^2$ and $(1 - z)^4$.

When $q \geq 1$, passing from $q - 1$ to q changes the binary digit sum by $1 - \nu_2(q)$. Hence $\varepsilon_q = -\varepsilon_{q-1}$ exactly when $\nu_2(q)$ is even. This proves the two factor criteria. Finally, the middle term in $\varepsilon_q + 6\varepsilon_{q-1} + \varepsilon_{q-2}$ has magnitude six while the other two have total magnitude at most two. $\square$

The first two strict negative half-integers are also never roots: $D_m(-1/2) = \varepsilon_m \neq 0$, and

$$D_m(-3/2) = \varepsilon_m - 2\varepsilon_{m-1} + \varepsilon_{m-2} \neq 0.$$

For $m \geq 2$, the second assertion would require three consecutive Thue–Morse signs to be equal, while every three consecutive positions contain an aligned pair $(2q, 2q + 1)$ of opposite signs. The cases $m = 0, 1$ are immediate from the same formula with the zero extension just stated.

The experiments included with this article suggest a stronger statement.

Conjecture 8.4 (No strict negative half-integer roots). *For all $m, r \in \mathbb{N}$,*

$$D_m\left(-r - \frac{1}{2}\right) \neq 0. \quad (88)$$

Equivalently, every negative rational root of D_m is an integer.

The accompanying exact Python program found no counterexample for every odd $1 \leq b \leq 63$ and every $0 \leq m \leq 10000$ in $D_m(-b/2)$. This finite computation is evidence only; the conjecture is not used elsewhere. Formula (82) converts the problem into a nonvanishing statement for parity-restricted binomial sums of consecutive Thue–Morse signs.

9 Fast exact evaluation

There are three distinct computational tasks, and they should not be confused:

1. generate the symbolic polynomial $D_m(x)$ for a fixed diagonal;
2. evaluate one near-edge value $D_m(n)$ when m is moderate;
3. evaluate an arbitrary table entry when $m = k - n - 1$ may be enormous.

The best formula is different in each regime.

9.1 Symbolic generation of any diagonal

For a single symbolic polynomial, (30) is direct and finite. The following Wolfram Language definition automatically replaces all manually entered diagonal rules:

```
tmSign[q_Integer?NonNegative] := (-1)^ThueMorse[q];

risingTwoX[x_, 0] := 1;
risingTwoX[x_, p_Integer?Positive] :=
  Product[2 x + j, {j, 0, p - 1}];

DiagonalPolynomial[m_Integer?NonNegative, x_] := Expand @ Sum[
  With[{p = m - 2 q}, tmSign[q] risingTwoX[x, p]/p!],
  {q, 0, Floor[m/2]}
];
```

For example,

```
Table[Factor[DiagonalPolynomial[m, x]], {m, 0, 20}]
```

generates the first twenty-one diagonal laws exactly. No calls to `InterpolatingPolynomial`, numerical fitting, or rational reconstruction are involved.

When a whole prefix $D_0, \dots, D_M$ is needed, the Newton recurrence (41) reuses previous work:

```
lambda[r_Integer?Positive, x_] :=
  2 x + 2 - 2^(IntegerExponent[r, 2] + 1);

diagonalFormal[0] = 1;
diagonalFormal[m_Integer?Positive] := diagonalFormal[m] = Expand[
  Sum[lambda[r, formalX] diagonalFormal[m - r], {r, 1, m}]/m
];
```

The two generators are algebraically independent implementations and are therefore useful for regression testing each other.

9.2 Direct exact evaluation

At an integer row n , the finite binomial formula is

$$D_m(n) = \sum_{q=0}^{\lfloor m/2 \rfloor} \varepsilon_q \binom{2n + m - 2q - 1}{m - 2q}. \quad (89)$$

For a small or moderate m this is excellent: it has only $\lfloor m/2 \rfloor + 1$ exact summands. The corresponding Wolfram Language routine is

```
DiagonalValue[m_Integer?NonNegative, n_Integer?NonNegative] := Sum[
  With[{p = m - 2 q},
    tmSign[q] Binomial[2 n + p - 1, p]
  ],
  {q, 0, Floor[m/2]}
];
```

The block law (62) first reduces an arbitrary m modulo $B = 2^{2n+1}$:

$$D_m(n) = \varepsilon_{\lfloor m/B \rfloor} D_{m \bmod B}(n). \quad (90)$$

Unlike the original code's block rule, this version acts directly on the natural diagonal offset. It avoids bookkeeping caused by the shift $n + 1$.

9.3 Logarithmic-time signed prefix moments

The direct sum is still unsuitable when the reduced offset is very large. The key observation is that, for fixed $n \geq 1$, its binomial kernel is a polynomial of degree $2n - 1$ in the Thue–Morse index q :

$$\binom{2n + m - 2q - 1}{m - 2q} = \binom{2n + m - 2q - 1}{2n - 1} = \sum_{d=0}^{2n-1} c_d(m, n) q^d. \quad (91)$$

Thus only finitely many signed prefix moments are required.

Definition 9.1 (Signed Thue–Morse prefix moments). For $N, d \in \mathbb{N}$, let

$$M_d(N) = \sum_{q=0}^{N-1} \varepsilon_q q^d. \quad (92)$$

The sum is exclusive at N ; $M_d(0) = 0$.

Theorem 9.2 (Binary divide-and-conquer recurrence). For $N, d \in \mathbb{N}$,

$$M_d(2N) = - \sum_{j=0}^{d-1} \binom{d}{j} 2^j M_j(N), \quad (93)$$

$$M_d(2N + 1) = M_d(2N) + \varepsilon_N (2N)^d. \quad (94)$$

The sum in (93) is empty when $d = 0$.

Proof. Pair the terms with indices $2q$ and $2q + 1$ and use (1):

$$\begin{aligned} M_d(2N) &= \sum_{q=0}^{N-1} \left(\varepsilon_{2q} (2q)^d + \varepsilon_{2q+1} (2q+1)^d \right) \\ &= \sum_{q=0}^{N-1} \varepsilon_q \left((2q)^d - (2q+1)^d \right) \\ &= - \sum_{j=0}^{d-1} \binom{d}{j} 2^j \sum_{q=0}^{N-1} \varepsilon_q q^j. \end{aligned}$$

The highest-degree terms cancel. Adding the final term at index $2N$ gives (94). $\square$

Let $L = \lfloor m/2 \rfloor + 1$. Combining (91) with (92) gives

$$D_m(n) = \sum_{d=0}^{2n-1} c_d(m, n) M_d(L) \quad (n \geq 1). \quad (95)$$

The recursion depth for $M_d(L)$ is $O(\log L)$. At each level, computing all moments through degree $2n - 1$ by (93) uses $O(n^2)$ arithmetic operations. Ignoring integer multiplication bit costs, the evaluator therefore uses

$$O(n^2 \log m) \quad (96)$$

exact arithmetic operations, rather than $O(m)$ summands.

Algorithm 9.3 (Hybrid evaluator). Given $n, k \in \mathbb{N}$:

1. If $k \leq n$, return 0.

2. Set $m = k - n - 1$, $B = 2^{2n+1}$, and write $m = qB + r$.
3. Compute the core value $D_r(n)$ by the direct sum if r is below a chosen small cutoff; otherwise use (95).
4. Return $\varepsilon_q D_r(n)$.

The cutoff only chooses between two exact algorithms; it does not affect the result. The supplied implementation uses 128 by default.

```
tmPrefixMoments[0, maxDegree_Integer?NonNegative] :=
  ConstantArray[0, maxDegree + 1];

tmPrefixMoments[length_Integer?Positive, maxDegree_Integer?NonNegative] :=
  tmPrefixMoments[length, maxDegree] = Module[
    {half = Quotient[length, 2], lower, paired},
    lower = tmPrefixMoments[half, maxDegree];
    paired = Table[
      If[d == 0, 0,
        -Sum[Binomial[d, j] 2^j lower[[j + 1]], {j, 0, d - 1}]],
      {d, 0, maxDegree}
    ];
    If[OddQ[length],
      paired += tmSign[half] Table[(2 half)^d, {d, 0, maxDegree}]]
    ];
  paired
];
```

For $n \geq 1$, the polynomial coefficients $c_d(m, n)$ are obtained exactly from

$$\frac{1}{(2n-1)!} \prod_{j=1}^{2n-1} (m - 2q + j).$$

The full source file `diagonal_polynomials.wl` contains this contraction, block reduction, a compact replacement for the original `s`, and a literal repeated-prefix reference implementation.

9.4 Memory and implementation notes

The memoized moment routine stores vectors keyed by the pair $(N, d_{\max})$. A single query touches only the binary ancestors $N, \lfloor N/2 \rfloor, \lfloor N/4 \rfloor, \dots$, so its live recursion depth is logarithmic. For a long batch with many unrelated degree bounds, one may remove memoization or cache moments at a fixed maximum degree and truncate as needed.

Exact integer sizes can be very large. Complexity (96) counts arithmetic operations, not bit operations; this is the appropriate distinction because the output itself may contain thousands or millions of bits. The algorithm avoids artificial work proportional to the index m , but it cannot avoid the cost of storing the exact answer.

The local recurrence (12) remains preferable when a dense rectangular region is desired. The moment method is designed for sparse random access to isolated entries. The diagonal polynomial and Newton–Bell methods are designed for symbolic dependence on n . Keeping these use cases separate leads to simpler and faster code.

10 Connection with the Fabius and Rvachev approximation layer

The diagonal theory is purely discrete, but it lands exactly on the finite polynomials used in the repository’s Fabius–Rvachev approximants. The Lean formalization proves that the order- k iterated

prefix coefficients below the first dyadic cutoff are the coefficients of the approximation polynomial p_{k-1} , and that division by $2^{\binom{k-1}{2}}$ gives the corresponding step-approximant values at cell centers [9].

For the odd order $k = 2n + 1$ considered here, the finite approximation polynomial is precisely

$$p_{2n}(z) = H_{2n}(z) = \prod_{j=1}^{2n} (1 + z + \cdots + z^{2^j-1}). \quad (97)$$

Therefore, whenever $0 \leq m < 2^{2n+1}$,

$$D_m(n) = [z^m]p_{2n}(z), \quad (98)$$

and the normalized step height is

$$\frac{D_m(n)}{2^{\binom{2n}{2}}}. \quad (99)$$

The normalizing denominator is exactly the plateau height C in (61). Thus the complement law (67) becomes a normalized reflection around $1/2$, and the central plateau (68) becomes the level 1.

For a fixed diagonal offset m , combine (38) with $C = 2^{n(2n-1)}$ to obtain

$$\frac{D_m(n)}{2^{\binom{2n}{2}}} = \frac{2^m n^m}{m!} 2^{-n(2n-1)} \left(1 + \frac{m(m-1)}{4n} + \mathcal{O}(n^{-2}) \right). \quad (100)$$

Hence every fixed number of coefficient cells from the left edge is suppressed by a quadratic exponential in n , up to a polynomial factor. This is an exact quantitative boundary-layer statement for the finite approximants. It is consistent with the infinite-order flatness of the limiting Fabius/Rvachev functions, but it should not be confused with an asymptotic at a fixed spatial argument: the cell represented by a fixed m moves toward the support endpoint as the grid is refined.

The block factorization also clarifies why the early repeated-summation picture sees the Fabius shape at all. After division by $H_{2n}(1)$, the coefficients of H_{2n} are the probability mass function of a sum of independent discrete uniforms on $\{0, \dots, 2^j - 1\}$, while the normalized limit is related to the continuous sum of geometrically scaled uniforms appearing in the probability model of Rvachev's up-function. The present article does not reprove that analytic limit; it isolates the exact finite coefficient algebra on which that limit acts.

11 Further deductions and research directions

The closed form (30) resolves the basic diagonal question, but it also exposes several directions that are not visible from the first eight polynomials.

11.1 Scaling regimes beyond fixed diagonals

The fixed- m asymptotic (38) is only the extreme left edge. There are at least four natural regimes for $m = m(n)$:

1. m fixed;
2. $m \rightarrow \infty$ but $m = o(2^n)$, where only the first few geometric upper bounds in (57) are active;
3. m proportional to the standard deviation of the restricted-uniform sum, where a local central limit theorem should control $h_{2n,m}$;
4. m within $O(n)$ of the exact plateau and zero-run boundaries, where finite edge corrections dominate.

A uniform saddle-point analysis of $H_{2n}(z)$ could connect the polynomial edge law to the Gaussian bulk and the exact flat plateau without passing through separate approximations.

11.2 Residual factors and zero geometry

After removing the completely classified nonnegative rational factor R_m^+ and the negative integer factors detected by (82), the remaining factor often appears irreducible over $\mathbb{Q}$ in low degrees. Questions include:

- Are all nonnegative half-integer zeros simple?
- Can the negative integer roots be characterized by binary automata in the pair (m, r) from the binomial sums at $x = -r$?
- Does [theorem 8.4](#) hold?
- What is the limiting distribution of the nonrational complex zeros of D_m after an m -dependent scaling?

The half-step lowering law suggests that finite-difference analogues of classical results on Appell and Sheffer zero interlacing may be relevant, but $\mathcal{T}(z^2)$ is not a positivity-preserving prefactor, so standard real-rooted arguments do not apply directly.

11.3 Automatic and p -adic structure

The cumulants (40) depend on r only through $\nu_2(r)$. This makes the Newton recurrence an unusual hybrid of Bell-polynomial and automatic-sequence structure. It is natural to ask for:

- finite automata or linear representations for coefficient residues of the primitive polynomials Q_m modulo odd primes;
- exact Newton polygons of Q_m at 2 beyond the coarse bound (75);
- analogues for generalized base- b Thue–Morse products $\prod_{j \geq 0} (1 - z^{b^j})$ or for weighted digit characters.

For a general digit character, the logarithmic cumulants are controlled by b -adic divisibility, so much of the Riordan and Bell machinery survives with ν_2 replaced by ν_b and with a different finite block.

11.4 Formalization opportunities

The repository already formalizes the inclusive-prefix convolution, dyadic zero runs, finite approximation polynomials, and corrected normalization. The following additions are comparatively self-contained candidates for Lean formalization:

1. the shifted-table identity (6);
2. the diagonal generating function (26) and finite formula (30);
3. the half-step recurrence (45);
4. the finite-block theorem (55);
5. primitive normalization and the exact denominator theorem;
6. the rational-root restriction;
7. the signed-prefix-moment recurrences and correctness of the fast evaluator.

Most proofs are finite formal-power-series or elementary integer-divisibility arguments and should integrate naturally with the existing modules on Thue–Morse generating series and Prouhet cancellation.

12 Summary of the main formulas

For quick reference, the complete computational chain is:

$$\begin{aligned}
\varepsilon_q &= (-1)^{s_2(q)}, \\
\mathcal{T}(z) &= \sum_{q \geq 0} \varepsilon_q z^q = \prod_{j \geq 0} (1 - z^{2^j}), \\
s(n, k) &= 0 \quad (k \leq n), \\
s(n, k) &= D_{k-n-1}(n) \quad (k > n), \\
\sum_{m \geq 0} D_m(x) z^m &= \frac{\mathcal{T}(z^2)}{(1 - z)^{2x}}, \\
D_m(x) &= \sum_{q=0}^{\lfloor m/2 \rfloor} \varepsilon_q \frac{(2x)^{\overline{m-2q}}}{(m-2q)!}, \\
m D_m(x) &= \sum_{r=1}^m \left(2x + 2 - 2^{\nu_2(r)+1} \right) D_{m-r}(x), \\
D_m(x) - D_m(x - \tfrac{1}{2}) &= D_{m-1}(x), \\
Q_m(x) &= \frac{m!}{2^{\lceil m/2 \rceil}} D_m(x) \in \mathbb{Z}[x] \quad \text{and is primitive,} \\
\text{den}(D_m) &= \frac{m!}{2^{\min(\nu_2(m!), \lceil m/2 \rceil)}}.
\end{aligned}$$

For $a \geq 0$, with $B_a = 2^{a+1}$,

$$D_m(a/2) = 0 \iff m \bmod B_a \geq B_a - a - 1.$$

For $b \geq 0$,

$$D_m(-b/2) = \sum_{\substack{0 \leq j \leq \min(b, m) \\ j \equiv m \pmod{2}}} (-1)^j \binom{b}{j} \varepsilon_{(m-j)/2}.$$

For arbitrary random access, expand $\binom{2n+m-2q-1}{2n-1}$ as a degree- $(2n-1)$ polynomial in q and contract it with the recursively computed signed moments $M_d(\lfloor m/2 \rfloor + 1)$. This completes the passage from a finite list of hand-entered diagonal rules to a general symbolic and algorithmic theory.

A Nonnegative rational factors through $m = 31$

The following table lists $\mathcal{A}_m$ from (78). The displayed squarefree product $\prod_{a \in \mathcal{A}_m} (2x - a)$ contains one copy of every nonnegative rational linear factor; an empty set means that D_m has no nonnegative rational zero.

m	$\mathcal{A}_m$	m	$\mathcal{A}_m$
0	$\emptyset$	16	$\emptyset$
1	$\{0\}$	17	$\{0\}$
2	$\{1\}$	18	$\{1\}$
3	$\{0, 1\}$	19	$\{0, 1\}$
4	$\emptyset$	20	$\emptyset$
5	$\{0, 2\}$	21	$\{0, 2\}$
6	$\{1, 2\}$	22	$\{1, 2\}$

m	$\mathcal{A}_m$	m	$\mathcal{A}_m$
7	$\{0, 1, 2\}$	23	$\{0, 1, 2\}$
8	$\emptyset$	24	$\emptyset$
9	$\{0\}$	25	$\{0\}$
10	$\{1\}$	26	$\{1\}$
11	$\{0, 1\}$	27	$\{0, 1, 4\}$
12	$\{3\}$	28	$\{3, 4\}$
13	$\{0, 2, 3\}$	29	$\{0, 2, 3, 4\}$
14	$\{1, 2, 3\}$	30	$\{1, 2, 3, 4\}$
15	$\{0, 1, 2, 3\}$	31	$\{0, 1, 2, 3, 4\}$

The table makes the nested terminal windows before powers of two visible. At $m = 2^{a+1} - a - 1, \dots, 2^{a+1} - 1$, the new root $x = a/2$ appears; lower-period root patterns continue independently underneath it.

B Complete Wolfram Language source

The following listing is the exact source distributed as `diagonal_polynomials.wl`.

```
(* ::Package:: *)

(*
  Exact Wolfram Language implementation for the diagonal-polynomial theory
  of odd iterated Thue--Morse prefix sums.

  Definitions used in the accompanying article:

    epsilon_q = (-1)^ThueMorse[q]
    D_m(x)    = Coefficient of z^m in T[z^2]/(1-z)^(2 x)

  and, for the table in the question,

    s[n,k] = 0                                when k <= n,
    s[n,k] = D_-(k-n-1)(n)                    when k > n.

  Everything below is exact. No interpolation or numerical recognition is
  used. The direct formula is normally the fastest choice for one diagonal
  value; the Newton--Bell generator is convenient when many symbolic
  polynomials are wanted.
*)

ClearAll[
  tmSign, risingTwoX, DiagonalPolynomial, DiagonalValue,
  tmPrefixMoments, DiagonalValueByMoments, DiagonalValueFast,
  $DiagonalDirectCutoff, SClosed, lambda, diagonalFormal,
  DiagonalPolynomialNewton, SReference, VerifyDiagonalCode,
  formalX
];

(* Signed Thue--Morse sequence epsilon_q in {+1,-1}. *)
tmSign[q_Integer?NonNegative] := (-1)^ThueMorse[q];

(* (2 x)^(overline p), written as a literal polynomial product so that
   Expand always produces a polynomial without needing FunctionExpand. *)
risingTwoX[x_, 0] := 1;
risingTwoX[x_, p_Integer?Positive] := Product[2 x + j, {j, 0, p - 1}];

(* Exact polynomial for the m-th diagonal:

    D_m(x) = Sum[epsilon_q (2x)^(overline(m-2q))/(m-2q)!, q].
  *)
DiagonalPolynomial[m_Integer?NonNegative, x_] := Expand @ Sum[
```

```

With[{p = m - 2 q}, tmSign[q] risingTwoX[x, p]/p!],
{q, 0, Floor[m/2]}
];

(* Exact integer value D_m(n). Generalized Binomial also handles n=0;
   in particular Binomial[-1,0] is 1. *)
DiagonalValue[m_Integer?NonNegative, n_Integer?NonNegative] := Sum[
  With[{p = m - 2 q},
    tmSign[q] Binomial[2 n + p - 1, p]
  ],
  {q, 0, Floor[m/2]}
];

(* Signed prefix moments

    M_d(N) = Sum[epsilon_q q^d, {q,0,N-1}].

    Pairing 2q and 2q+1 gives

    M_d(2N) = -Sum[Binomial[d,j] 2^j M_j(N), {j,0,d-1}],
    M_d(2N+1) = M_d(2N) + epsilon_N (2N)^d.

    Consequently the entire vector through degree d is computed in
    O(d^2 Log[N]) exact arithmetic operations. *)
tmPrefixMoments[0, maxDegree_Integer?NonNegative] :=
  ConstantArray[0, maxDegree + 1];

tmPrefixMoments[length_Integer?Positive, maxDegree_Integer?NonNegative] :=
  tmPrefixMoments[length, maxDegree] = Module[
    {half = Quotient[length, 2], lower, paired},
    lower = tmPrefixMoments[Quotient[length, 2], maxDegree];
    paired = Table[
      If[d == 0,
        0,
        -Sum[Binomial[d, j] 2^j lower[[j + 1]], {j, 0, d - 1}]
      ],
      {d, 0, maxDegree}
    ];
    If[OddQ[length],
      paired += tmSign[half] Table[(2 half)^d, {d, 0, maxDegree}]
    ];
    paired
  ];

(* Logarithmic-in-m evaluator. For n>=1 the binomial kernel is a
   polynomial of degree 2n-1 in q, so it can be contracted with the signed
   prefix moments instead of summed term by term. *)
DiagonalValueByMoments[m_Integer?NonNegative, 0] :=
  If[EvenQ[m], tmSign[m/2], 0];

DiagonalValueByMoments[m_Integer?NonNegative, n_Integer?Positive] := Module[
  {degree = 2 n - 1, q, kernel, coefficients, moments, value},
  kernel = Expand[
    Product[m - 2 q + j, {j, 1, 2 n - 1}]/(2 n - 1)!
  ];
  coefficients = CoefficientList[kernel, q];
  moments = tmPrefixMoments[Floor[m/2] + 1, degree];
  value = Together[coefficients . Take[moments, Length[coefficients]]];
  If[Denominator[value] == 1, Numerator[value], value]
];

(* Exact block reduction in the diagonal offset m. For

    B = 2^(2 n + 1), m = q B + r,

    one has D_m(n) = epsilon_q D_r(n). A short reduced offset is handled by
    the direct finite sum; a long one uses the prefix-moment algorithm. *)
$DiagonalDirectCutoff = 128;

DiagonalValueFast[m_Integer?NonNegative, n_Integer?NonNegative] := Module[
  {block = 2^(2 n + 1), q, r, core},
  {q, r} = QuotientRemainder[m, block];

```

```

core = If[r <= $DiagonalDirectCutoff,
  DiagonalValue[r, n],
  DiagonalValueByMoments[r, n]
];
tmSign[q] core
];

(* A compact replacement for the whole two-dimensional definition. *)
SClosed[n_Integer?NonNegative, k_Integer?NonNegative] /; k <= n := 0;
SClosed[n_Integer?NonNegative, k_Integer?NonNegative] /; k > n :=
  DiagonalValueFast[k - n - 1, n];

(* The 2-adic logarithmic cumulants


$$\lambda_r(x) = 2x + 2^{-2^{\text{IntegerExponent}[r, 2] + 1}}.$$


*)
lambda[r_Integer?Positive, x_] :=
  2 x + 2^{-2^{\text{IntegerExponent}[r, 2] + 1}};

(* Memoized Newton--Bell generator. It obeys


$$m D_m(x) = \text{Sum}[\lambda_r(x) D_{m-r}(x), \{r, 1, m\}].$$


*)
diagonalFormal[0] = 1;
diagonalFormal[m_Integer?Positive] := diagonalFormal[m] = Expand[
  Sum[lambda[r, formalX] diagonalFormal[m - r], {r, 1, m}]/m
];

DiagonalPolynomialNewton[m_Integer?NonNegative, x_] :=
  diagonalFormal[m] /. formalX -> x;

(* Literal reference implementation through (2 n + 1) inclusive prefix
  summations. This is for verification, not for fast production use. *)
SReference[n_Integer?NonNegative, k_Integer?NonNegative] /; k <= n := 0;
SReference[n_Integer?NonNegative, k_Integer?NonNegative] /; k > n := Module[
  {last = k - n - 1, row},
  row = Nest[Accumulate, Table[tmSign[j], {j, 0, last}], 2 n + 1];
  row[[last + 1]]
];

(* Small exact regression suite. True means that the direct formula,
  Newton--Bell recurrence, half-step lowering law, and literal prefix sums
  all agree on the tested range. *)
VerifyDiagonalCode[maxDegree_Integer?NonNegative : 10] := And[
  And @@ Table[
    Expand[
      DiagonalPolynomial[m, formalX] -
      DiagonalPolynomialNewton[m, formalX]
    ] == 0,
    {m, 0, maxDegree}
  ],
  And @@ Table[
    Expand[
      DiagonalPolynomial[m, formalX] -
      (DiagonalPolynomial[m, formalX] /. formalX -> formalX - 1/2) -
      DiagonalPolynomial[m - 1, formalX]
    ] == 0,
    {m, 1, maxDegree}
  ],
  And @@ Flatten @ Table[
    DiagonalValue[m, n] == DiagonalValueByMoments[m, n],
    {n, 0, 4}, {m, 0, 40}
  ],
  And @@ Flatten @ Table[
    SClosed[n, k] == SReference[n, k],
    {n, 0, 4}, {k, 0, 28}
  ]
];

(* Examples:


$$\text{Table}[\text{Factor}[\text{DiagonalPolynomial}[m, x]], \{m, 0, 12\}]$$


```

```

DiagonalValue[40, 1000]
SClosed[20, 10^9]      (uses the prefix-moment path)
VerifyDiagonalCode[10]

The user's displayed rule for a chosen offset m can be produced by

With[{m = 7},
  HoldForm[s[n_Integer, k_Integer] /; k == n + m + 1] ==
    Factor[DiagonalPolynomial[m, n]]
]
*)

```

C Complete Python verification source

The Python program uses SymPy exact arithmetic. It verifies the direct and Newton–Bell polynomial generators, the half-step law, literal repeated prefix summation, finite-block reduction, prefix-moment evaluation, primitive normalization, exact denominators, and both half-grid formulas. It also contains the finite scan reported after [theorem 8.4](#).

```

#!/usr/bin/env python3
"""Exact experiments for diagonal polynomials in iterated Thue--Morse sums.

The article defines

    epsilon(q) = (-1) ** popcount(q),
    D_m(x) = [z^m] T(z^2) / (1-z)^(2x),
    T(z) = product_{j>=0} (1-z^(2^j)).

For an integer row n and original column k,

    s(n,k) = 0                                if k <= n,
    s(n,k) = D_{k-n-1}(n)                    if k > n.

This script performs exact symbolic and integer checks of the formulas in the
article. It deliberately uses only exact SymPy arithmetic; no floating-point
recognition is used anywhere.

Typical use:

    python diagonal_polynomials.py --max-degree 12 --verify

The verification bounds are intentionally modest so that the script runs
quickly on ordinary hardware. Increase them in 'run_verification' when a
larger experimental sweep is desired.
"""

from __future__ import annotations

import argparse
import math
from functools import lru_cache
from typing import Iterable

import sympy as sp

X = sp.Symbol("x")

def thue_morse_sign(n: int) -> int:
    """Return epsilon_n = (-1)^popcount(n) for n >= 0."""
    if n < 0:
        raise ValueError("the Thue--Morse index must be nonnegative")
    return -1 if n.bit_count() & 1 else 1

def rising_two_x(p: int, x: sp.Expr = X) -> sp.Expr:
    """Return (2*x)^(overline p), with the empty product equal to 1."""
    if p < 0:

```

```

        raise ValueError("the rising-factorial order must be nonnegative")
    return sp.rf(2 * x, p)

@lru_cache(maxsize=None)
def diagonal_polynomial(m: int) -> sp.Expr:
    r"""Return the exact polynomial D_m(x).

    The finite rising-factorial formula is

        D_m(x) = sum_{q=0}^{floor(m/2)} epsilon_q
                  (2*x)^{overline{m-2q}} / (m-2q)!.

    This is generally faster and more stable than interpolating values of the
    original recursively defined two-dimensional table.
    """
    if m < 0:
        return sp.Integer(0)
    terms: list[sp.Expr] = []
    for q in range(m // 2 + 1):
        p = m - 2 * q
        terms.append(
            thue_morse_sign(q) * rising_two_x(p) / sp.factorial(p)
        )
    return sp.Poly(sum(terms), X, domain=sp.QQ).as_expr()

def diagonal_value(m: int, n: int) -> int:
    r"""Evaluate D_m(n) by the finite generalized-binomial formula.

        D_m(n) = sum_q epsilon_q binomial(2n+m-2q-1, m-2q).

    SymPy correctly interprets binomial(-1, 0)=1, so the formula also covers
    n=0 without a special case.
    """
    if m < 0:
        return 0
    if n < 0:
        raise ValueError("this evaluator expects a nonnegative integer row")
    value = sp.Integer(0)
    for q in range(m // 2 + 1):
        p = m - 2 * q
        value += thue_morse_sign(q) * sp.binomial(2 * n + p - 1, p)
    return int(value)

def diagonal_value_block_reduced(m: int, n: int) -> int:
    r"""Evaluate D_m(n) after the exact Thue--Morse block reduction.

    Let B=2^(2n+1). The finite-product factorization gives

        D_m(n) = epsilon_floor(m/B) D_(m mod B)(n).

    This is useful when m is enormous but n is small. It reduces the direct
    binomial sum to fewer than B/2 terms, independently of the magnitude of m.
    """
    if m < 0:
        return 0
    if n < 0:
        raise ValueError("the row must be nonnegative")
    block = 1 << (2 * n + 1)
    q, r = divmod(m, block)
    return thue_morse_sign(q) * diagonal_value(r, n)

@lru_cache(maxsize=None)
def thue_morse_prefix_moments(length: int, max_degree: int) -> tuple[int, ...]:
    r"""Return M_d(length)=sum_{q < length} epsilon_q q^d for 0<=d<=max_degree.

    Pairing q=2u and q=2u+1 gives the divide-and-conquer recurrences

        M_d(2N) = -sum_{j<d} binomial(d,j) 2^j M_j(N),

```

```

    M_d(2N+1) = M_d(2N) + epsilon_N (2N)^d.

The recursion has depth  $O(\log \text{length})$  and computes the whole moment vector
in  $O(\text{max\_degree}^2 \log \text{length})$  exact arithmetic operations.
"""
if length < 0 or max_degree < 0:
    raise ValueError("length and max_degree must be nonnegative")
if length == 0:
    return tuple(0 for _ in range(max_degree + 1))

half = length // 2
lower = thue_morse_prefix_moments(half, max_degree)
paired: list[int] = []
for degree in range(max_degree + 1):
    value = -sum(
        math.comb(degree, j) * (2 ** j) * lower[j]
        for j in range(degree)
    )
    paired.append(value)

if length & 1:
    sign = thue_morse_sign(half)
    for degree in range(max_degree + 1):
        paired[degree] += sign * ((2 * half) ** degree)

return tuple(paired)

def diagonal_value_by_moments(m: int, n: int) -> int:
    r"""Evaluate  $D_m(n)$  in time polynomial in  $n$  and logarithmic in  $m$ .

    For  $n \geq 1$ , write the generalized binomial in the direct formula as the
    degree- $(2n-1)$  polynomial

        
$$\text{binomial}(2n+m-2q-1, 2n-1) = \sum_d c_d q^d.$$


    Then  $D_m(n) = \sum_d c_d M_d(\text{floor}(m/2)+1)$ , where  $M_d$  is computed by
    ‘‘thue_morse_prefix_moments‘‘. This avoids a loop with  $O(m)$  summands.
    """
    if m < 0:
        return 0
    if n < 0:
        raise ValueError("the row must be nonnegative")
    if n == 0:
        return thue_morse_sign(m // 2) if m % 2 == 0 else 0

    degree = 2 * n - 1
    q = sp.Symbol("q")
    p = m - 2 * q
    kernel = sp.prod(p + j for j in range(1, 2 * n)) / math.factorial(degree)
    kernel_poly = sp.Poly(sp.expand(kernel), q, domain=sp.QQ)
    moments = thue_morse_prefix_moments(m // 2 + 1, degree)
    value = sum(kernel_poly.nth(d) * moments[d] for d in range(degree + 1))
    if value.q != 1:
        raise ArithmeticError("an integer-valued formula produced a noninteger")
    return int(value)

def diagonal_value_hybrid(m: int, n: int, direct_cutoff: int = 128) -> int:
    r"""Block-reduce  $m$ , then choose a short sum or the moment algorithm."""
    if m < 0:
        return 0
    if n < 0 or direct_cutoff < 0:
        raise ValueError("n and direct_cutoff must be nonnegative")
    block = 1 << (2 * n + 1)
    q, r = divmod(m, block)
    core = (
        diagonal_value(r, n)
        if r <= direct_cutoff
        else diagonal_value_by_moments(r, n)
    )
    return thue_morse_sign(q) * core

```



```

@lru_cache(maxsize=None)
def diagonal_polynomial_newton(m: int) -> sp.Expr:
    r"""Generate D_m(x) from the 2-adic Newton--Bell recurrence.

    Put

        lambda_r(x) = 2x + 2 - 2^(v_2(r)+1).

    Then D_0=1 and

        m D_m = sum_{r=1}^m lambda_r D_{m-r}.
    """
    if m < 0:
        return sp.Integer(0)
    if m == 0:
        return sp.Integer(1)
    total = sp.Integer(0)
    for r in range(1, m + 1):
        valuation = (r & -r).bit_length() - 1
        lam = 2 * X + 2 - 2 ** (valuation + 1)
        total += lam * diagonal_polynomial_newton(m - r)
    return sp.Poly(total / m, X, domain=sp.QQ).as_expr()

def iterated_prefix_row(order: int, last_index: int) -> list[int]:
    """Compute S^(order)(0..last_index) by literal inclusive prefix sums."""
    if order < 0 or last_index < 0:
        raise ValueError("order and last_index must be nonnegative")
    row = [thue_morse_sign(j) for j in range(last_index + 1)]
    for _ in range(order):
        running = 0
        next_row: list[int] = []
        for value in row:
            running += value
            next_row.append(running)
        row = next_row
    return row

def s_from_definition(n: int, k: int) -> int:
    """Literal reference implementation of the user's s(n,k)."""
    if n < 0 or k < 0:
        raise ValueError("n and k must be nonnegative")
    if k <= n:
        return 0
    prefix_index = k - n - 1
    return iterated_prefix_row(2 * n + 1, prefix_index)[prefix_index]

def expected_common_denominator(m: int) -> int:
    r"""Return the theorem's exact common denominator of D_m.

        den(D_m) = m! / 2^min(v_2(m!), ceil(m/2)).
    """
    factorial = math.factorial(m)
    v2_factorial = 0
    t = factorial
    while t and t % 2 == 0:
        v2_factorial += 1
        t //= 2
    c = (m + 1) // 2
    return factorial // (2 ** min(v2_factorial, c))

def actual_common_denominator(poly: sp.Expr) -> int:
    """Least common multiple of the reduced coefficient denominators."""
    denominators = [int(c.q) for c in sp.Poly(poly, X, domain=sp.QQ).all_coeffs()]
    return math.lcm(*denominators) if denominators else 1

```

```

def nonnegative_half_root_predicted(m: int, a: int) -> bool:
    r"""Test the exact residue criterion for  $D_m(a/2)=0$ .

    With  $B=2^{a+1}$ , the root occurs exactly when  $m \bmod B$  is one of the final
     $a+1$  residues  $B-a-1, \dots, B-1$ .
    """
    if m < 0 or a < 0:
        raise ValueError("m and a must be nonnegative")
    block = 1 << (a + 1)
    return (m % block) >= block - a - 1

def negative_half_value(m: int, b: int) -> int:
    r"""Evaluate  $D_m(-b/2)$  by the finite binomial/Thue--Morse formula.

    
$$D_m(-b/2) = \sum_{0 \leq j \leq \min(b,m)} (-1)^j \binom{b}{j} \epsilon_{(m-j)/2}$$

    """
    if m < 0 or b < 0:
        raise ValueError("m and b must be nonnegative")
    total = 0
    for j in range(min(b, m) + 1):
        if (m - j) % 2 == 0:
            total += (
                (-1) ** j
                * math.comb(b, j)
                * thue_morse_sign((m - j) // 2)
            )
    return total

def scan_negative_strict_half_roots(max_odd_b: int, max_m: int) -> list[tuple[int, int]]:
    r"""Search for zeros  $D_m(-b/2)$  with odd b (strict negative halves).

    The article records the conjecture that this list is always empty. This
    function is only an exact finite experiment; it is not used as a proof.
    """
    if max_odd_b < 1 or max_m < 0:
        raise ValueError("max_odd_b must be positive and max_m nonnegative")
    zeros: list[tuple[int, int]] = []
    for b in range(1, max_odd_b + 1, 2):
        for m in range(max_m + 1):
            if negative_half_value(m, b) == 0:
                zeros.append((b, m))
    return zeros

def primitive_normalization(m: int) -> sp.Poly:
    r"""Return  $Q_m(x)=m! D_m(x)/2^{\lceil m/2 \rceil}$ , a primitive  $\mathbb{Z}[x]$  polynomial."""
    c = (m + 1) // 2
    expr = sp.factorial(m) * diagonal_polynomial(m) / (2 ** c)
    return sp.Poly(expr, X, domain=sp.ZZ)

def print_polynomials(max_degree: int) -> None:
    r"""Print factored diagonal polynomials and their exact denominators."""
    for m in range(max_degree + 1):
        poly = diagonal_polynomial(m)
        print(
            f"D_{m}(x) = {sp.factor(poly)}"
            f"      [common denominator {actual_common_denominator(poly)}]"
        )

def run_verification(max_degree: int) -> None:
    r"""Run a collection of exact regression checks from the article."""
    symbolic_bound = max(max_degree, 10)

    # Closed form versus Newton--Bell generation.
    for m in range(symbolic_bound + 1):
        assert sp.expand(diagonal_polynomial(m) - diagonal_polynomial_newton(m)) == 0

```

```

# The Sheffer lowering law:  $D_m(x) - D_m(x-1/2) = D_{m-1}(x)$ .
for m in range(1, symbolic_bound + 1):
    lhs = diagonal_polynomial(m) - diagonal_polynomial(m).subs(X, X - sp.Rational(1, 2))
    assert sp.expand(lhs - diagonal_polynomial(m - 1)) == 0

# Direct formula versus literal repeated prefix summation. Build each
# literal row once instead of recomputing all lower prefixes for every k.
for n in range(5):
    largest_k = 34
    largest_prefix_index = largest_k - n - 1
    literal = iterated_prefix_row(2 * n + 1, largest_prefix_index)
    for k in range(0, largest_k + 1):
        if k <= n:
            observed = 0
        else:
            observed = literal[k - n - 1]
        expected = 0 if k <= n else diagonal_value(k - n - 1, n)
        assert observed == expected

# Block reduction in the diagonal offset m. A sparse collection of edge,
# center, and multi-block samples avoids spending most of the run time on
# enormous generalized binomial sums while still exercising every case.
for n in range(5):
    block = 1 << (2 * n + 1)
    residues = {0, 1, 2, n, n + 1, block // 4, block // 2, block - n - 2, block - 1}
    samples = {q * block + r for q in range(5) for r in residues if 0 <= r < block}
    for m in sorted(samples):
        direct = diagonal_value(m, n)
        assert direct == diagonal_value_block_reduced(m, n)
        assert direct == diagonal_value_by_moments(m, n)
        assert direct == diagonal_value_hybrid(m, n, direct_cutoff=16)

# Additional moderately large offsets exercise the logarithmic-time
# prefix-moment recursion without making the literal direct sum dominant.
for n, m in [(2, 10_003), (5, 1_000_007), (10, 10**9 + 7)]:
    assert diagonal_value_hybrid(m, n, direct_cutoff=16) == \
        diagonal_value_hybrid(m + (1 << (2 * n + 1)), n, direct_cutoff=16) * \
        thue_morse_sign(1)

# Exact common-denominator and primitive-content theorems.
for m in range(symbolic_bound + 1):
    assert actual_common_denominator(diagonal_polynomial(m)) == expected_common_denominator(m)
    primitive = primitive_normalization(m)
    content, _ = sp.polys.polytools.primitive(primitive)
    assert abs(int(content)) == 1

# Exact nonnegative half-integer zero criterion.
for m in range(symbolic_bound + 1):
    for a in range(0, 12):
        value = diagonal_polynomial(m).subs(X, sp.Rational(a, 2))
        assert (value == 0) == nonnegative_half_root_predicted(m, a)

# Negative half-grid evaluation formula.
for m in range(symbolic_bound + 1):
    for b in range(0, 12):
        value = diagonal_polynomial(m).subs(X, -sp.Rational(b, 2))
        assert value == negative_half_value(m, b)

print(
    "All exact checks passed: closed forms, Newton recurrence, half-step "
    "lowering, repeated sums, block reduction, logarithmic-time moment evaluation, denominators,
    and half-grid values."
)

def parse_args(argv: Iterable[str] | None = None) -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument(
        "--max-degree",
        type=int,
        default=10,
        help="largest D_m to print (default: 10)",
    )

```

```

    )
    parser.add_argument(
        "--verify",
        action="store_true",
        help="run exact symbolic and integer regression checks",
    )
    parser.add_argument(
        "--scan-negative-half",
        action="store_true",
        help="scan the article's strict-negative-half-root conjecture",
    )
    parser.add_argument(
        "--scan-max-b",
        type=int,
        default=63,
        help="largest odd b in the conjecture scan (default: 63)",
    )
    parser.add_argument(
        "--scan-max-m",
        type=int,
        default=10_000,
        help="largest diagonal m in the conjecture scan (default: 10000)",
    )
    return parser.parse_args(argv)

def main() -> None:
    args = parse_args()
    if args.max_degree < 0:
        raise SystemExit("--max-degree must be nonnegative")
    print_polynomials(args.max_degree)
    if args.verify:
        run_verification(args.max_degree)
    if args.scan_negative_half:
        zeros = scan_negative_strict_half_roots(args.scan_max_b, args.scan_max_m)
        print(
            f"Strict-negative-half scan through odd b <= {args.scan_max_b} "
            f"and m <= {args.scan_max_m}: {len(zeros)} zero(s)."
        )
        if zeros:
            print(zeros[:50])

if __name__ == "__main__":
    main()

```

D Reproduction commands

From the directory containing the source files:

```

python diagonal_polynomials.py --max-degree 12 --verify
python diagonal_polynomials.py --max-degree 3 --scan-negative-half
latexmk -pdf -interaction=nonstopmode -halt-on-error \
    thue_morse_diagonal_polynomials.tex

```

The first command performs the theorem-level regression checks and prints factored polynomials. The second performs the exact finite conjecture scan through odd $b \leq 63$ and $m \leq 10000$. The LaTeX source uses only files included in the archive.

References

- [1] J.-P. Allouche and J. Shallit, The ubiquitous Prouhet–Thue–Morse sequence, in *Sequences and Their Applications: Proceedings of SETA '98*, C. Ding, T. Helleseth, and H. Niederreiter, eds.,

- Springer Series in Discrete Mathematics and Theoretical Computer Science, Springer, London, 1999, pp. 1–16.
- [2] J.-P. Allouche and J. Shallit, *Automatic Sequences: Theory, Applications, Generalizations*, Cambridge University Press, Cambridge, 2003.
 - [3] L. Comtet, *Advanced Combinatorics: The Art of Finite and Infinite Expansions*, D. Reidel, Dordrecht, 1974.
 - [4] L. W. Shapiro, S. Getu, W.-J. Woan, and L. C. Woodson, The Riordan group, *Discrete Applied Mathematics* **34** (1991), 229–239, [https://doi.org/10.1016/0166-218X\(91\)90088-E](https://doi.org/10.1016/0166-218X(91)90088-E).
 - [5] R. Sprugnoli, Riordan arrays and combinatorial sums, *Discrete Mathematics* **132** (1994), 267–290.
 - [6] S. Roman, *The Umbral Calculus*, Pure and Applied Mathematics, vol. 111, Academic Press, New York, 1984.
 - [7] V. Reshetnikov, *K-fold summation over the signed Thue–Morse sequence*, early research note in the ProveIt repository, Analysis/FabiusFunction/docs/archive/early-notes/, file object 837bdf2b6711388b83d2e65c92824aa1f3e4c48d, accessed 3 September 2026.
 - [8] V. Reshetnikov, FabiusFunction/ThueMorsePrefix.lean, formal development of inclusive iterated prefixes, convolution, and zero runs in the ProveIt repository, file object 1c6fe79e3a71fb2f06efd6817aa497aa9aa3753e, accessed 3 September 2026.
 - [9] V. Reshetnikov, FabiusFunction/ThueMorseApproximation.lean, formal development of finite approximation polynomials, corrected normalization, and step approximants in the ProveIt repository, file object 5678201a0060b9cf91ef3fb2995c647b73b5f423, accessed 3 September 2026.
 - [10] V. Reshetnikov, FabiusFunction/DraftCounterexamples.lean, machine-checked audit of false proxy, normalization, and error claims in an early draft, in the ProveIt repository, file object 6de3948507c382c95521f6de768bc43ba7a8081f, accessed 3 September 2026.